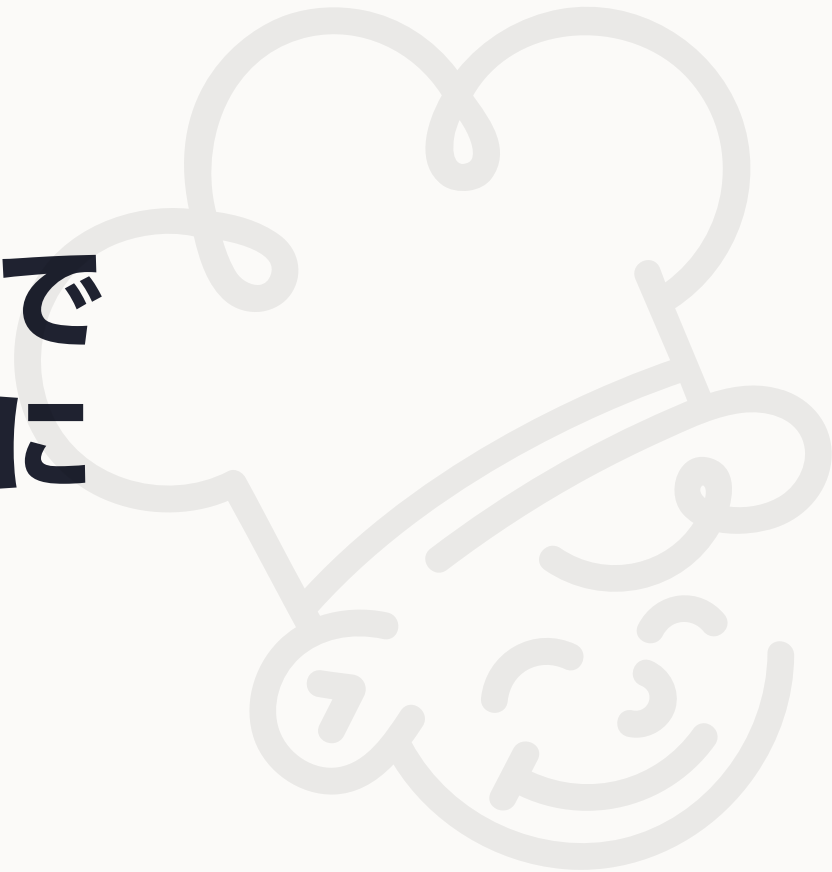


mise bootstrapで dotfilesを宣言的に

@newt239



| newt @newt239

- 芝浦工業大学 B3
- Webフロントエンドエンジニア
- 興味領域
 - デザインシステム
 - Webアクセシビリティ
 - フロントエンドエコシステム
 - Web標準



| mise bootstrap

- mise の設定ファイルにマシンの状態を宣言し、コマンド 1 つで反映
- 対象はパッケージ・dotfiles・macOS の設定・ログインシェル・ツール
- v2026.6.6 で experimental 機能として追加

The screenshot shows the 'mise-en-place' website. The header includes the logo, a search bar, and navigation links for 'mise-versions', 'Dev Tools', 'Environments', 'Tasks', and 'v2026.10.4'. A sidebar on the left lists navigation items: 'Start Here', 'Getting Started', 'Demo', 'Walkthrough', 'Installing mise', 'IDE Integration', 'Continuous Integration', 'Configuration', and 'mise.toml'. The main content area is titled 'Bootstrap' and contains the following text: 'mise bootstrap applies the machine setup declared in your mise configuration: packages, files, services, repositories, shell setup, tools, and a final task. Use it for workstation or server setup that needs more than installing [tools]. Run it explicitly when you want to apply that configuration.' Below this, it says 'Start with the parts your machine needs, preview them, and add more resources as the configuration grows. Each section has its own status and apply'. On the right, there is a 'On this page' section with links to 'Example', 'Starting from a repository', 'A bootstrap project', 'Global mise configuration', 'Shared dotfile history', 'How it runs', 'Previewing changes', 'Inspecting state', and 'What does where'.

Before: Makefile + シェルスクリプト

- `make` で `init` → `link` → `defaults` → `brew` の順にスクリプトを実行
- 処理の手順を記述する手続き型。再実行時の挙動はスクリプトごとに異なる

```
dotfiles/  
├─ Makefile  
├─ .bin/  
│   ├── init.sh  
│   ├── link.sh  
│   ├── mise.sh  
│   ├── brew.sh  
│   ├── defaults.sh  
│   └─ Brewfile  
├─ editor/  
│   └─ vscode.sh  
└─ home/  
    └─ .mise.toml
```

```
all: init link defaults brew  
  
init:  
    @.bin/init.sh  
link:  
    @.bin/link.sh  
defaults:  
    @.bin/defaults.sh  
brew:  
    @.bin/brew.sh
```

シンボリックリンク

- `symlink-each` で `home/` 以下をファイル単位でリンク
- `home/` 以外に置いたファイルはリンク先ごとにリンク元を指定

Before `.bin/link.sh`

```
link() {  
  mkdir -p "$(dirname "$2")"  
  ln -fns "$REPO/$1" "$2"  
}  
  
cd "$REPO/home"  
find . -type f ! -name .DS_Store |  
  while read -r f; do  
    link "home/${f#./}" "$HOME/${f#./}"  
  done  
  
link config/gnupg/gpg-agent.conf \  
  "$HOME/.gnupg/gpg-agent.conf"
```

After `home/.mise.toml`

```
[dotfiles."~"]  
source = "~/dotfiles/home"  
mode = "symlink-each"  
exclude = [".DS_Store"]  
  
[dotfiles."~/.gnupg/gpg-agent.conf"]  
source = "~/dotfiles/config/gnupg/gpg-agent.conf"
```

| macOS の設定

- Dock・Finder・キーボード・トラックパッドは専用のキーで記述
- ネストした plist は `defaults_entries` の `path` で該当する値だけを変更

Before `.bin/defaults.sh`

```
defaults write com.apple.dock autohide -bool true
defaults write com.apple.finder AppleShowAllFiles -bool true
defaults write com.apple.symbolichotkeys.plist AppleSymbolicHotKeys -dict-add 64 "
    <dict>
      <key>enabled</key><false/>
      <key>value</key><dict>
        <key>type</key><string>standard</string>
        <key>parameters</key>
        <array>
          <integer>65535</integer>
          <integer>49</integer>
          <integer>1048576</integer>
        </array>
      </dict>
    </dict>
"
```

After `home/.mise.toml`

```
[bootstrap.macos.dock]
autohide = true

[bootstrap.macos.finder]
show_all_files = true

[[bootstrap.macos.defaults_entries]]
domain = "com.apple.symbolichotkeys"
key = "AppleSymbolicHotKeys"
path = ["64", "enabled"]
value = false
```

パッケージ

- `brew:` `brew-cask:` `mas:` のプレフィックスでインストール元を指定
- `mise.<環境名>.toml` に分割し、`-E <環境名>` で読み込むファイルを切り替え

Before `.bin/Brewfile`

```
cask "google-chrome"
cask "visual-studio-code"
brew "mise"
cask "docker-desktop"
cask "ghostty"

brew "curl"
brew "direnv"
brew "gh"
brew "postgresql@17", restart_service: true, link: true
brew "mas"
```

After `home/.mise.toml` / `home/.mise.work.toml`

```
[bootstrap.packages]
"brew:curl" = "latest"
"brew:direnv" = "latest"
"brew:gh" = "latest"
"brew:postgresql@17" = "latest"
"brew:mas" = "latest"

"brew-cask:google-chrome" = "latest"
"brew-cask:visual-studio-code" = "latest"
"brew-cask:docker-desktop" = "latest"
"brew-cask:ghostty" = "latest"
"mas:1429033973" = "latest"
```

ログインシェルとツール

- ログインシェルも宣言の対象
- `[tools]` のインストールは `bootstrap` の手順の 1 つとして実行

Before `.bin/init.sh` / `.bin/mise.sh`

```
if [ "$SHELL" != "/bin/zsh" ]; then
    chsh -s /bin/zsh
fi
```

```
source ~/.zshrc
if [ ! -f "$HOME/.mise.toml" ]; then
    echo ".mise.tomlファイルが見つかりません。"
    exit 1
fi
mise install
```

After `home/.mise.toml`

```
[bootstrap.user]
login_shell = "/bin/zsh"

[tools]
go = "1.26.1"
python = "3.14.3"
rust = "1.94.0"
node = "24.21.0"
```

| mise bootstrap status

- 宣言した項目ごとに現在の状態を一覧表示
- `differs` は宣言と実際の状態が一致していない項目

```
$ mise bootstrap status -C home -E work
dotfiles  ~                               symlink-each ~/dotfiles/home    differs
dotfiles  ~/.gnupg/gpg-agent.conf symlink ~/dotfiles/config/gnupg/gpg-agent.conf  applied
defaults  com.apple.dock autohide                true      set
defaults  com.apple.dock persistent-apps       array (7 items) differs
defaults  NSGlobalDomain InitialKeyRepeat        10        set
defaults  NSGlobalDomain com.apple.keyboard.fnState false      differs
user      login_shell                       /bin/zsh  set
tools     node@24.21.0                             24.21.0   installed
packages  brew:gh                                  2.100.0   installed
packages  brew-cask:ghostty                       1.3.1     installed (auto-updates)
```

導入方法

- mise v2026.7.4 以降は experimental 設定なしで利用可能
- `mise.toml` に `[dotfiles]` や `[bootstrap.*]` を記述

手順	コマンド
mise のインストール	<code>brew install mise</code>
設定ファイルを信頼済みとして登録	<code>mise trust</code>
変更内容の確認	<code>mise bootstrap plan</code>
反映	<code>mise bootstrap</code>

そのほかの機能

dotfiles の履歴と巻き戻し

`track` したファイルは変更ごとにチェックポイントを記録

```
# 記録と巻き戻し
mise bootstrap dotfiles track ~/.zshrc
mise bootstrap dotfiles history
mise bootstrap dotfiles rollback ~/.zshrc

# 複数マシンで共有
mise bootstrap dotfiles origin set <Git URL>
mise bootstrap dotfiles sync
mise bootstrap dotfiles pull
```

モジュール単位の適用と撤去

`mise.<名前>.toml` 単位で選択し、`unapply` で撤去

```
# mise.ssh.toml と mise.gpg.toml もあわせて反映
mise -E ssh,gpg bootstrap
# mise.ssh.toml で宣言したものを撤去
mise bootstrap unapply ssh
```

mise bootstrapでdotfilesを宣言的に

既存の環境から宣言を作成

インストール済みの Homebrew の formula を取り込み

```
mise bootstrap packages import
```

SSH 越しの適用

OpenSSH 経由で複数のマシンに bootstrap を実行

```
mise bootstrap remote --host user@server
```

常駐プロセスの宣言

LaunchAgent や systemd のサービスとして常駐

```
[bootstrap.services.postgresql]
scope = "user"
command = "/opt/homebrew/opt/postgresql@17/bin/postgres"
environment = { PGDATA = "/opt/homebrew/var/postgresql@17" }
```

| 参考

- [Bootstrap | mise-en-place](#)
- [続・dotfiles管理、miseに全部任せてみた ～mise の最新 bootstrap 機能を追う～](#)
- [newt239/dotfiles](#)